

Chapter Four:Communication

Compiled By:Bekretsyon B.(Msc)

April 7, 2025

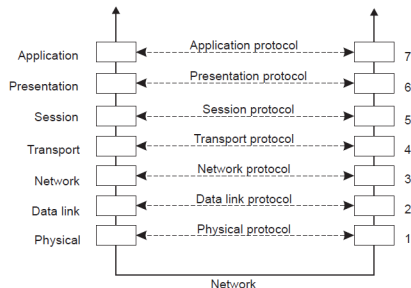
Outline

- Layered Communication Protocols
 - Layered Protocols
 - Types of Communication
- Remote Procedure Call (RPC)
 - Basic RPC operation
 - Parameter passing
 - Extended RPC Models
- Message-Oriented Communication
 - Transient Messaging Models:
 - Persistent Messaging Model
- Stream-Oriented Communication
 - Continuous media
 - Stream and Quality of service
 - Stream synchronization
- Multicast communication
 - Introduction
 - Application-level multicasting
 - Gossip-based data dissemination

Layered Protocols

- Communications are often handled by **layered protocols**.
- Protocols are formal set of rules that govern the layered protocols **format, contents**, and **meaning** of the messages sent and received.
- Agreements and specifications are needed at each level.
- Two general types of protocols:
 - **Connection oriented protocols**
 - Do some initial work to set up a "virtual circuit".
 - **Connection less protocols**
 - Do not set up a virtual circuit.
- Examples:
 - Phone is connection-oriented.
 - Mail is connectionless.
- Pros and Cons
 - Connection-oriented are usually more efficient.
 - Connectionless are usually more efficient for one-off messages

Basic Networking Model: Basic Networking Model



Drawbacks

- Focus on message-passing only
- Often unneeded or unwanted functionality
- Violates access transparency

Layered Protocols: Low-Level Layers

Recap

- Physical layer
 - Contains the specification and implementation of bits, and their transmission between sender and receiver.
- Data link layer
 - Prescribes the transmission of a series of bits into a **frame**.
 - As physical layers are unreliable, main job of this layer is to **detect and correct errors**.
- Network layer
 - Describes how packets in a network of computers are to be **routed**.
- For many distributed systems, the **lowest level interface** is that of the **network layer**.

Layered Protocols: Transport Layer

Important

- The transport layer provides the **actual communication facilities** for most distributed systems.

Standard Internet protocols

- TCP: connection-oriented, reliable, stream-oriented communication
- UDP: unreliable (best-effort) datagram communication

Note

- IP multicasting is often considered a standard available service (which may be dangerous to assume).

Layered Protocols: Application Layer

Recap

- Session layer
 - Dialog control, checkpoints for long transfers
- Presentation layer
 - Meaning of bits, define records, etc.
- Application layer
 - Protocols for things like mail, file transfer, communication terminals.
 - Examples:
 - FTP (File Transfer Protocol)
 - HTTP (HyperText Transfer Protocol)
- Session and presentation layers are not effectively used and in practice only the application layer is ever used.

Layered Protocols: Middleware Layer

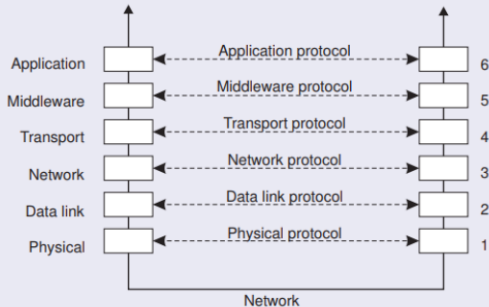
- Middleware is logically at application layer.
- Middleware is invented to provide **common** services and protocols that can be used by many **different** applications
 - A rich set of **communication protocols**
 - **Marshaling/Unmarshaling** of data, necessary for integrated systems
 - **Naming protocols**, to allow easy sharing of resources
 - **Security protocols** for secure communication
 - **Scaling mechanisms**, such as for replication and caching

Note

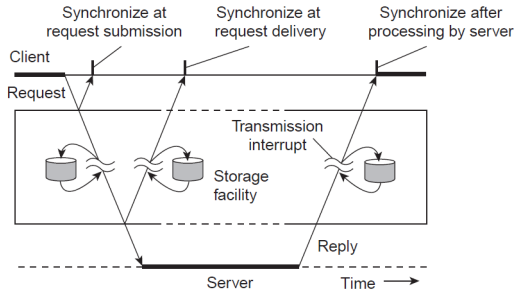
- What remains are truly application-specific protocols...

Layered Protocols: Middleware Layer

An adapted reference model for networked communication



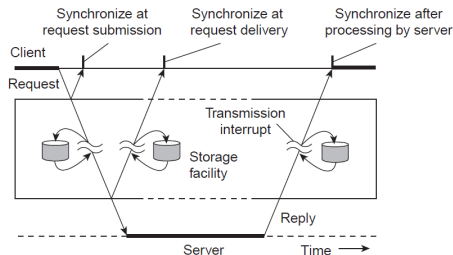
Types of communication



Distinguish

- Transient versus persistent communication
- Asynchronous versus synchronous communication

Types of communication



Transient versus persistent

- **Transient communication:** A message is discarded by a communication server as soon as it cannot be delivered at the next server, or at the receiver.
- **Persistent communication:** A message is stored at a communication server as long as it takes to deliver it at the receiver.

Types of communication

Asynchronous versus Synchronous

- **Asynchronous**: a sender continues immediately after it has submitted its message for transmission
- **Synchronous**: the sender is blocked until its message is stored in a local buffer at the receiving host or delivered to the receiver

Places for synchronization

- At request submission
- At request delivery
- After request processing

Types of communication

Client/Server

- Client/Server computing is generally based on a model of **transient synchronous communication**
 - Client and server have to be active at time of communication.
 - Client issues request and blocks until it receives reply
 - Server essentially waits only for incoming requests, and subsequently processes them
- Drawbacks synchronous communication
 - Client cannot do any other work while waiting for reply
 - Failures have to be handled immediately: the client is waiting
 - The model may simply not be appropriate (mail, news)

Messaging

Messaging

Message-oriented middleware aims at high-level **persistent asynchronous communication**:

- Processes send each other messages, which are queued
- Sender need not wait for immediate reply, but can do other things
- Middleware often ensures fault tolerance

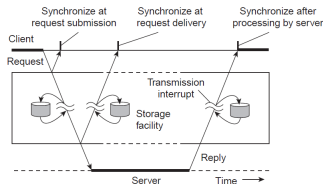
Remote Procedure Call (RPC)

- Basic RPC operation
- Parameter passing
- Extended RPC Models

Basic RPC operation

Observations

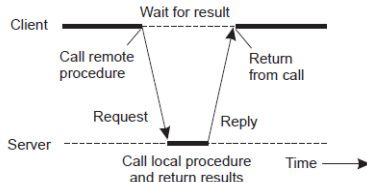
- Application developers are familiar with simple procedure model
- Well-engineered procedures operate in isolation (black box)
- There is no fundamental reason not to execute procedures on separate machine



- **Conclusion:** Communication between caller and callee can be hidden by using procedure-call mechanism.

Basic RPC operation

- RPC is a normal looking procedure call processed on a remote machine.
- What needs to happen?
 - Pack parameters and other information into a message ([marshalling](#)).
 - Send message to process on remote machine.
 - Unpack message on remote machine ([unmarshalling](#)).
 - Call the appropriate remote procedure, locally.
 - Get the return value, send back.



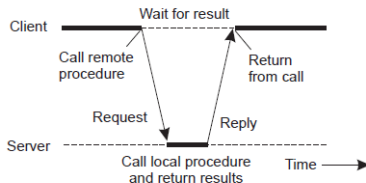
Basic RPC operation: Stubs

■ Client stub

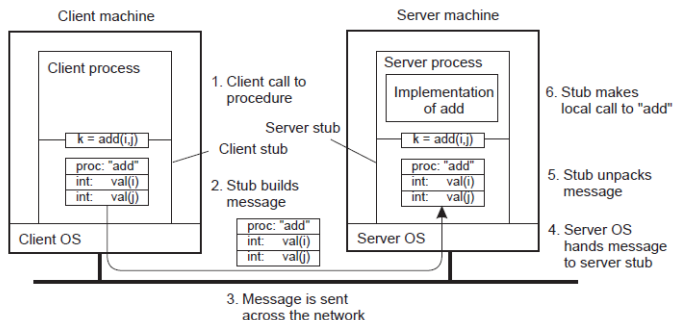
- Piece of code responsible for **proxying** the remote call as a local call, and packing the call up as a message.

■ Server stub

- Piece of code responsible for **unpacking** the message on the server, and **invoking** the actual server-side, application-level implementation.



Basic RPC operation: Steps of a Remote Procedure Call



- 1 Client procedure calls client stub.
- 2 Stub builds message; calls local OS.
- 3 OS sends message to remote OS.
- 4 Remote OS gives message to stub.
- 5 Stub unpacks parameters and calls server.

- 6 Server makes local call and returns result to stub.
- 7 Stub builds message; calls OS.
- 8 OS sends message to client's OS.
- 9 Client's OS gives message to stub.
- 10 Client stub unpacks result and returns to the client.

RPC: Parameter passing

Ways of Passing Parameters

- By value and By reference

Parameter marshaling

- There's more than just wrapping parameters into a message:
 - Wrapping a parameter means **transforming a value into a sequence of bytes**
 - The machines may differ in data representation (e.g., IBM mainframes use EBCDIC whereas IBM PCs use ASCII)
 - Client and server machines may have **different data representations** (think of byte ordering)
 - Client and server have to **agree on the same encoding**:
 - How are **basic data values represented** (integers, floats, characters)
 - How are **complex data values represented** (arrays, unions)
 - Client and server need to **properly interpret messages**, transforming them into machine-dependent representations.

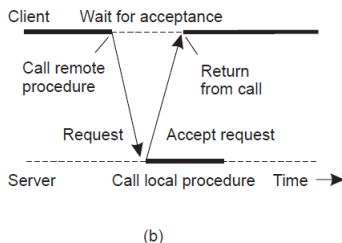
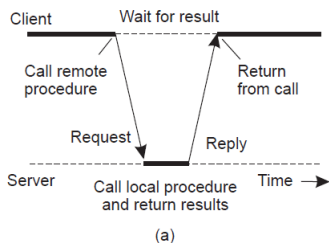


RPC: Parameter passing

- Assumption
 - Copy in/copy out semantics: while procedure is executed, nothing can be assumed about parameter values.
 - All data that is to be operated on is passed by parameters.
Excludes passing references to (global) data.
- Notes:
 - Full access transparency cannot be realized.
 - A remote reference mechanism enhances access transparency:
 - Remote reference offers unified access to remote data
 - Remote references can be passed as parameter in RPCs.
- RPC uses an Interface Definition Language (IDL) to describe an interface in a language-neutral way, enabling communication between software components that do not share a language.
- IDL is a specification language used to describe a software component's interface.
- IDL enables the machines at either end of the link to use different operating systems and computer languages, thus, offering a bridge between the two different systems.

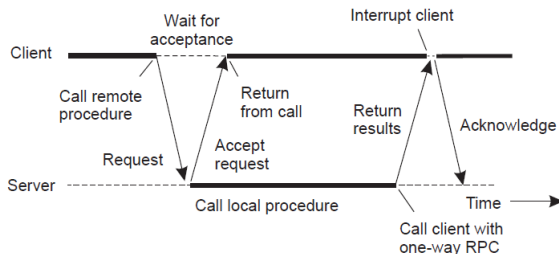
Extended RPC Models: Asynchronous RPC

Try to get rid of the strict request-reply behavior, but let the client continue without waiting for an answer from the server.



- a The interconnection between client and server in a traditional RPC
- b The interaction using asynchronous RPC

Extended RPC Models: Deferred Synchronous RPC



Variation

- Client can also do a (non)blocking poll at the server to see whether results are available.
- Can be thought of as interacting through **two asynchronous RPCs**.

Message-Oriented Communication

- Remote procedure calls and remote object invocations contribute to **hiding communication in distributed systems**, that is, they **enhance access transparency**.
- Example problems
 - In particular, when it cannot be assumed that the receiving side is executing at the time a request is issued, alternative communication services are needed.
 - The synchronous nature of RPCs, by which a client is blocked until its request has been processed, sometimes needs to be replaced by something else.
- That something else is messaging –; **Message-Oriented Communication**
 - **Transient Messaging Models**
 - **Persistent Messaging Models**

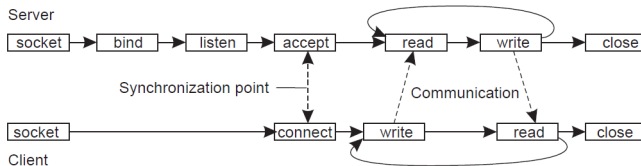
Transient Messaging Models: Sockets

- Many distributed systems and applications are built directly on top of the simple message-oriented model offered by the [transport layer](#)
- Standardizing the interface of transport layer helps to make use of the entire suite of messaging protocols through a simple set of primitives.
- [Berkeley Sockets](#) and [XTI \(X/Open Transport Interface\)](#) are examples of standard interfaces, with [Berkeley Sockets](#) being the most popular interface.
- Berkeley socket interface has been adopted by many systems including [Windows 2000/XP/Vista](#).
- A [socket](#) forms an abstraction over the actual communication port that is used by the local operating system for a specific transport protocol.
- Sockets provide [transient communication](#).

Transient messaging models: sockets

Socket Primitives for TCP/IP

SOCKET	Create a new communication endpoint
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept N connections
ACCEPT	Block until request to establish a connection
CONNECT	Attempt to establish a connection
SEND	Send data over a connection
RECEIVE	Receive data over a connection
CLOSE	Release the connection



Transient Messaging Models: Message-Passing Interface(MPI)

- Sockets are too **low level** for scientific computing.
 - No data types; No collective operations.
 - No message abstraction. Supporting only simple send and receive primitives.
- MPI was written to address such problems.
 - Offers **high-level** and **hardware independent** sets of primitives.
 - Provides communication among multiple **concurrent processes**.
 - Includes several varieties of **point-to-point communication**, as well as **collective communication** among groups of processes.
 - Implemented as **library of routines** which can be called from conventional programming languages.
 - Has been universally adopted by developers and users of parallel systems that rely on **message passing**. Includes **more than 125 functions**, with many different options and protocols.
 - **They were not designed for proprietary protocols developed for high-speed interconnection networks; portability will suffer**
- Like sockets, MPI provides **transient communication**.

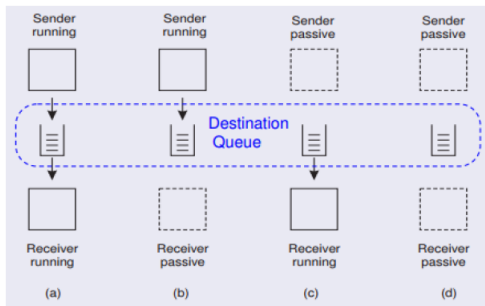
Transient Messaging Models: Message-Passing Interface (MPI)

Some of the Basic Message-Passing Operations of MPI

Primitive	Meaning
MPI_bsend	Send and wait till copied to local buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_sendrecv	Send a message and wait for reply
MPI_issend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there are none
MPI_irecv	Check if there is an incoming message, but do not block

Persistent Messaging Model: Message-Queuing System

- Message-queuing systems provide **persistent asynchronous**
- Communication through support of middleware-level **queues**.
- Queues correspond to **buffers** at communication servers.
- Four basic combinations for **loosely-coupled** communications using queues.



Four combinations for loosely-coupled communications using queues.

Persistent Messaging Model: Message-Queuing System

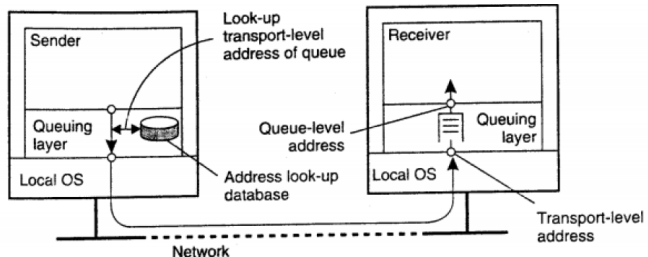
Basic Interface to a Queue in a Message-Queuing System

PUT	Append a message to a specified queue
GET	Block until the specified queue is nonempty, and remove the first message
POLL	Check a specified queue for messages, and remove the first. Never block
NOTIFY	Install a handler to be called when a message is put into the specified queue

Persistent Messaging Model: Message-Queuing System

General Architecture of a Message-Queuing System

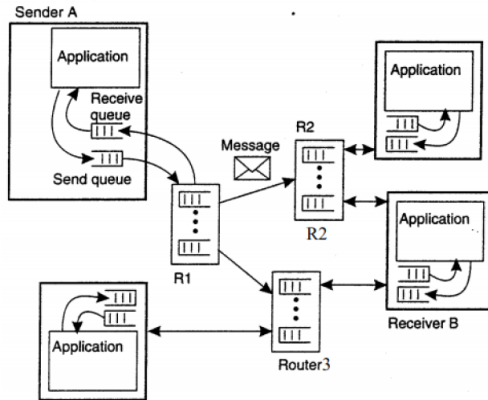
- Queues managed by queue managers.
- Some queue managers function as routers or relays, i.e., they forward incoming messages to other queue managers
- Queue names are generally low-level, transport-related.
 - Need to maintain another level of mapping.



Persistent Messaging Model: Message-Queuing System

The General Organization of a Message-Queuing System with Routers

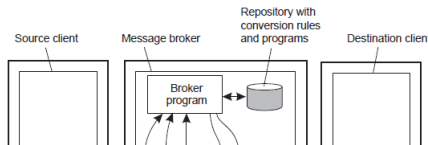
Assumption: A sends message to B



Persistent Messaging Model: Message-Queuing System

Message Brokers

- Message queuing systems assume a **common messaging protocol**: all applications agree on message format (i.e., structure and data representation)
- **Message broker** is a centralized component that takes care of application heterogeneity in a message-queuing system.
- **Message broker** functions higher-level than that typically done by a **router**.
 - **Converts** incoming messages to target format.
 - Acts as a kind of **application gateway**.
 - May provide **subject-based** routing capabilities.



Stream-Oriented Communication

- Continuous media
- Stream and Quality of service
- Stream synchronization

Support for Continuous Media

- All communication facilities discussed so far are essentially based on a **discrete**, that is **time-independent** exchange of information
- **Continuous media** are characterized by the fact that values are **time dependent**.
 - Audio
 - Video
 - Animations
 - Sensor data (temperature, pressure, etc.)
- Different timing with respect to data transfer results in different **transmission modes**.
 - **Asynchronous**- no restrictions with respect to when data is to be delivered
 - **Synchronous**- define a maximum end-to-end delay for individual data packets
 - **Isochronous**- define a maximum and minimum end-to-end delay (jitter is bounded)
- **Jitter** further refers to the variation in latency of packets carrying voice or video data over a communications channel.

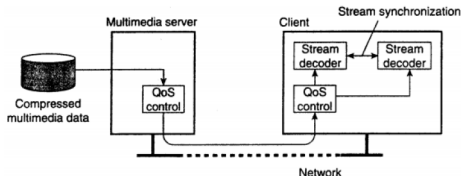
Support for Continuous Media

Data Stream

- A (**continuous**) **data stream** is a connection-oriented communication facility that supports **isochronous** data transmission.
- Some common stream characteristics:
 - Streams are **unidirectional**
 - There is generally a single **source**, and one or more **sinks**
 - Often, either the sink and/or source is a wrapper around hardware (e.g., camera, TV monitor, dedicated storage)
- Stream types:
 - **Simple**: consists of a single flow of data, e.g., audio or video
 - **Complex**: multiple data flows, e.g., stereo audio or combination of audio/video (may also contain subtitles, translation to other languages, ...)

Streams and Quality of Service

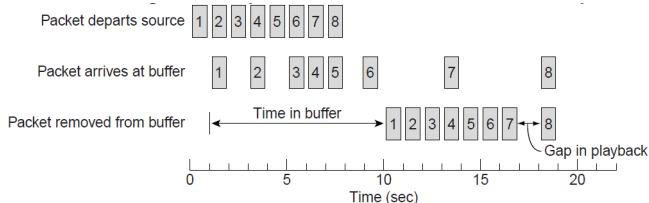
- Streams are all about timely delivery of data. How do you specify this **Quality of Service(QoS)**?
- Basics of **Quality of Service(QoS)** :
 - The required **bit rate** at which data should be transported.
 - The **maximum delay** until a session has been set up (i.e., when an application can start sending data).
 - The **maximum end-to-end delay** (i.e., how long it will take until a data unit makes it to a recipient).
 - The maximum delay variance, or **jitter**.
 - The maximum **round-trip delay**.
- A general architecture for streaming stored multimedia data over a network:



Streams and Quality of Service: Enforcing Quality of Service

Enforcing Quality of Service

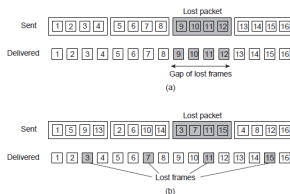
- There are various network-level tools, such as **differentiated services** by which certain packets can be prioritized.
- A distributed system can also help in getting data across to receivers.
- The one that is particularly useful is to use **buffers** to reduce jitter:



Streams and Quality of Service: Enforcing Quality of Service

Problem

How to reduce the effects of packet loss (when multiple samples are in a single packet)?



The effect of packet loss in (a) a non-interleaved (b) an interleaved transmission

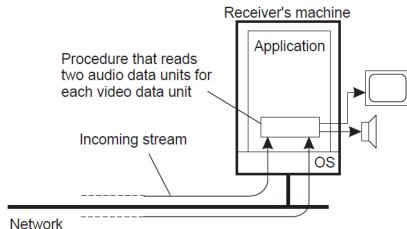
Requesting the sender to retransmit a missing packet is generally out of the question, so that **forward error correction (FEC)** needs to be applied

Stream synchronization

Problem

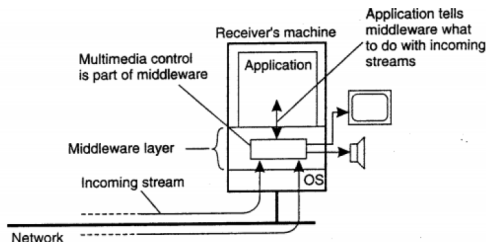
Given a complex stream, how do you keep the different substreams in synch?

- Think of playing out two channels, that together form stereo sound. Difference should be less than 20–30 msec!
- Synchronization at the lowest level
 - Synchronization is done explicitly by operating on the data units of simple streams.



Stream synchronization

- Synchronization at the **higher level**
 - Synchronization is done by the **middleware**, and controlled by the application using high-level interfaces.
 - **Multiplex** all substreams into a single stream, and **demultiplex** at the receiver.
 - Synchronization is handled at **multiplexing/demultiplexing** point.



The principle of synchronization as supported by high-level interfaces

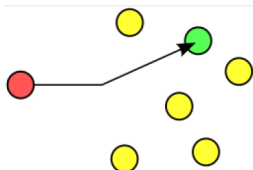
Multicast communication

- Introduction
- Application-level multicasting
- Gossip-based data dissemination

Introduction

Unicast Communication

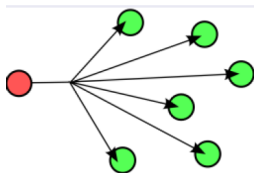
- Unicast is a **one-to-one** connection between the server and the client.
- Messages are sent to a **single network destination** identified by a unique address.
- Unicast-based media servers open and provide a stream for each unique user.
- Not efficient for **mass-distributed applications** since each network connection requires its own separate network bandwidth for transmission.



Introduction

Broadcast Communication

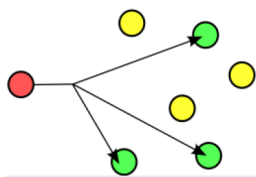
- Broadcast is a **one-to-all** connection in a given address range.
- Same message is transmitted to all possible destinations.



Introduction

Multicast Communication

- Multicast is a **one-to-many** connection in a given address range.
- Allows for communication that resembles a **conference call** where anyone from anywhere can join the conference, and everyone at the conference hears what the speaker says.
- Each client that listens to the multicast adds **no additional overhead** on the server



The END

Any Question?

Names, Identifiers, and Addresses

- A **name** in a distributed system is a string of bits or characters that is used to refer to an **entity**
- An **entity** is anything; e.g., resources such as hosts, printers, disks, files, objects, processes, users, Web pages, ...
- Entities can be **operated** on; e.g., a resource such as a printer offers an interface containing operations for printing a document, requesting the status of a job, ...
- To operate on an entity, it is necessary to access it through its **access point**, itself an entity (special)